LAPORAN PRAKTIKUM

STRUKTUR DATA

“ADT”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “ADT” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya ADT (Abstract Data Type) pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, krirtik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java

DAFTAR ISI

Table of Contents

KATA PENGANTAR ..... **i**
DAFTAR ISI **2**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II **3**
2.1 Langkah Kerja Praktikum 3
2.2 Analisis Hasil dan Pembahasan 10
BAB III **12**
3.1 Kesimpulan 12
DAFTAR PUSTAKA **13**
KATA PENGANTAR **1**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
BAB II **2**
2.1 Kelas ADT Mobil 2
2.2 Driver 2
2.2 Kelas Driver 3
BAB III **6**
3.1 Kesimpulan 6

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, kita tidak hanya belajar cara membuat program yang berjalan, tetapi juga bagaimana menyusun data dan fungsi agar lebih terstruktur dan mudah digunakan. Salah satu konsep penting dalam hal ini adalah **Abstract Data Type (ADT)** atau Tipe Data Abstrak.

ADT memungkinkan kita untuk merepresentasikan suatu objek beserta operasi-operasi yang bisa dilakukan terhadap objek tersebut tanpa harus terlalu fokus pada detail implementasinya. Dengan menggunakan ADT, program menjadi lebih rapi, mudah dipahami, dan mudah dikembangkan.

Pada praktikum ini, ADT diterapkan dalam pembuatan objek **Jam**, yang terdiri dari jam, menit, dan detik. Melalui program ini, mahasiswa dapat memahami bagaimana cara mengelola data waktu serta melakukan berbagai operasi seperti perhitungan, perbandingan, dan manipulasi waktu menggunakan konsep ADT.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

- Memahami konsep **Abstract Data Type (ADT)** dalam pemrograman Java
- Mampu membuat dan mengimplementasikan ADT dalam bentuk class
- Memahami penggunaan atribut, konstruktor, dan method dalam class
- Mampu melakukan operasi terhadap objek, seperti perhitungan dan perbandingan
- Melatih penggunaan konsep **Object-Oriented Programming (OOP)** dalam program sederhana

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah:

- Mahasiswa dapat memahami cara membuat program yang lebih terstruktur dan rapi
- Mahasiswa dapat menerapkan konsep ADT dalam pemrograman Java

- Meningkatkan pemahaman tentang konsep dasar **OOP** seperti enkapsulasi dan modularisasi
- Membantu mahasiswa dalam mengelola dan memanipulasi data, khususnya data waktu
- Menjadi dasar untuk membuat program yang lebih kompleks di masa depan

BAB II
PEMBAHASAN

2.1 Langkah Kerja Praktikum

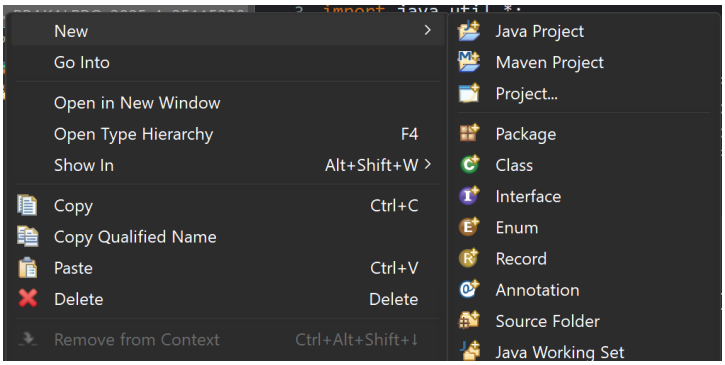
1. Persiapan Awal

   a) Buka aplikasi Eclipse IDE di laptop atau di komputer.

2. Membuat Project

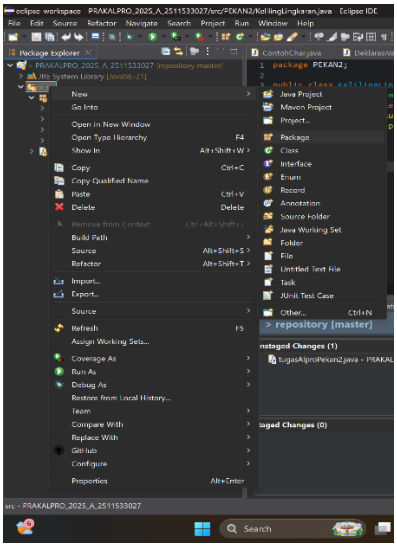
   a) Klik kanan pada bagian kosong disebelah kiri, klik **New**, lalu klik **New Project**.

   Beri nama Project **PrakSD2026_NIM**.

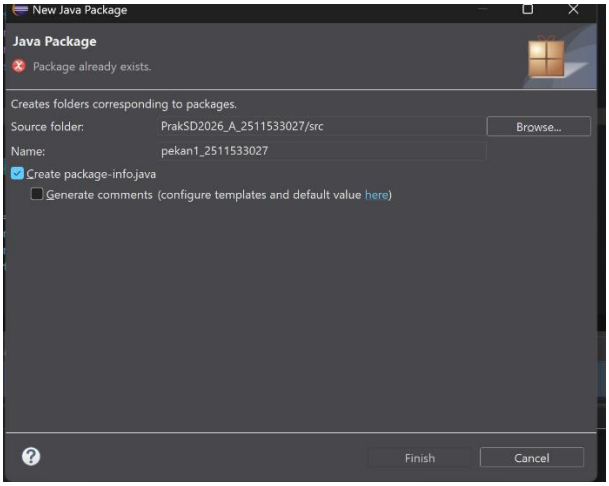


3. Membuat Package

   a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

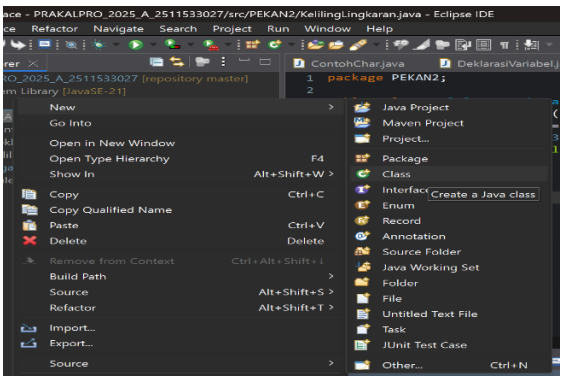


   b) Beri nama package **pekan1_NIM**.



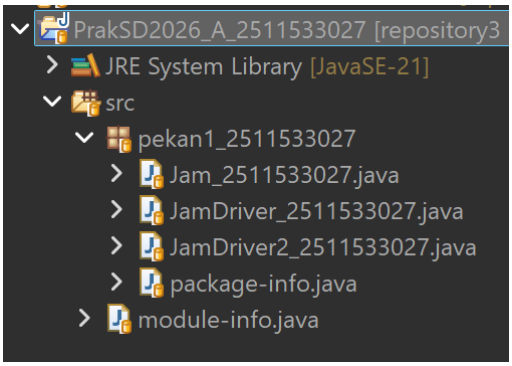
4. Membuat Class Baru untuk Setiap Percobaan

   a) Klik kanan pada package **pekan1_NIM**, lalu pilih **New**, terakhir klik **Class.**



   b) Buat beberapa kelas sesuai percobaan

      - Jam_NIM.java
      - JamDiver_NIM.java
      - JamDriver2_NIM.java



5. Menulis Kode Program

   a) Jam_NIM.java

      - Menuliskan program untuk merepresentasikan **waktu (jam, menit, detik)** dalam bentuk objek.
      - Kelas memiliki 3 atribut utama:

- ❖ hh untuk jam (0–23)
- ❖ mm untuk menit (0–59)
- ❖ ss untuk detik (0–59)

- Terdapat method **isValid(h, m, s)** untuk memvalidasi apakah nilai jam, menit, dan detik berada dalam rentang yang benar.
- Konstruktor Jam_2511533027(int h, int m, int s) digunakan untuk membuat objek jam dengan nilai tertentu.
- Method **getter (selektor)**:
  - ❖ getHour() untuk mengambil jam
  - ❖ getMinute() untuk mengambil menit
  - ❖ getSecond() untuk mengambil detik
- Method **setter (mutator)**:
  - ❖ setHour(h) untuk mengubah jam
  - ❖ setMinute(m) untuk mengubah menit
  - ❖ setSecond(s) untuk mengubah detik
- Method **toSeconds()** digunakan untuk mengonversi waktu menjadi total detik.
- Method **fromSeconds(total)** digunakan untuk mengubah total detik menjadi objek jam:
  - ❖ Jika detik negatif → error
  - ❖ Menggunakan konsep **wrap 24 jam** (biar waktu muter kayak jarum jam)
- Method **compareTo(other)** digunakan untuk membandingkan dua waktu berdasarkan jumlah detik.
- Method **equals(o)** digunakan untuk mengecek apakah dua objek jam memiliki nilai yang sama.
- Method **hashCode()** digunakan untuk menghasilkan nilai hash dari objek.
- Method **aritmatika waktu**:
  - ❖ plus(other) → menjumlahkan dua waktu
  - ❖ minus(other) → mengurangkan waktu (dengan wrap 24 jam)
  - ❖ nextSecond() → maju 1 detik
  - ❖ nextNSeconds(n) → maju n detik
  - ❖ prevSecond() → mundur 1 detik

- ❖ prevNSeconds(n) → mundur n detik
- Method **durasiDetik(jaw, jakh)** digunakan untuk menghitung selisih waktu dalam detik.
- Method **toString()** digunakan untuk menampilkan waktu dalam format: HH:MM:SS (08:05:29)

```java
package pekan1_2511533027;

public class Jam_2511533027 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    // --- Konstruktor/Kreator ---
    public Jam_2511533027(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m; }
    public void setSecond(int s) {this.ss = s; }

    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533027 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533027(h, m, s);
    }
```

```java
    // --- Relasional ---
    public int compareTo(Jam_2511533027 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533027 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int  hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511533027 plus(Jam_2511533027 other) { return fromSeconds(this.toSeconds() +  other.toSeconds()); }
    public Jam_2511533027 minus(Jam_2511533027 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*36
    public Jam_2511533027 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511533027 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511533027 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
    public Jam_2511533027 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }

    // --- Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi(JAw, JAkh)
    public static int durasiDetik(Jam_2511533027 jaw, Jam_2511533027 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
    //overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }

}
```

b) JamDriver_NIM.java

- Program ini digunakan sebagai **driver (penguji)** untuk menjalankan dan menguji kelas Jam_2511533027.
- Di dalam method main, dibuat dua objek waktu:
  - ❖ a = 23:59:50
  - ❖ b = 00:00:15
- Program menampilkan nilai awal dari kedua objek:
  - ❖ a ditampilkan menggunakan method toString()
  - ❖ b ditampilkan menggunakan method toString()
- Operasi **penjumlahan waktu**:
  - ❖ a.plus(b) digunakan untuk menjumlahkan waktu a dan b

- ❖ Hasil akan otomatis mengikuti sistem **wrap 24 jam**

- Operasi **penambahan detik**:
  - ❖ a.nextNSeconds(20) digunakan untuk menambahkan 20 detik ke waktu a
- Operasi **durasi waktu**:
  - ❖ Jam_2511533027.durasiDetik(a, b) digunakan untuk menghitung selisih waktu dari a ke b dalam satuan detik
- Operasi **perbandingan waktu**:
  - ❖ a.compareTo(b) digunakan untuk membandingkan waktu a dan b
  - ❖ Hasil:
    - Nilai negatif → a < b
    - Nilai 0 → a == b
    - Nilai positif → a > b
- Output ditampilkan ke layar menggunakan System.out.println() untuk setiap operasi yang dilakukan.

```java
package pekan1_2511533027;

public class JamDriver_2511533027 {
    public static void main(String[] args) {
        Jam_2511533027 a = new Jam_2511533027(23, 59, 50);
        Jam_2511533027 b = new Jam_2511533027(0, 0, 15);
        System.out.println("a        = " + a);
        System.out.println("b        = " + b);
        System.out.println("a+b      = " + a.plus(b));
        System.out.println("next 20s = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b) = " + Jam_2511533027.durasiDetik(a,b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }

}
```

Output:

```
Problems  Javadoc  Declaration  Console  Progress
<terminated> JamDriver2_2511533027 [Java Application] C:\Users\hp\.p2\pool\p
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)     : 12
Masukkan Menit (0-59    : 22
Masukkan Detik (0-59)   : 29

--- Input Jam 2 ---
Masukkan Jam (0-23)     : 09
Masukkan Menit (0-59    : 00
Masukkan Detik (0-59)   : 56

--- Hasil Operasi ---
Jam 1 (String)      : 12:22:29
Jam 2 (String)      : 09:00:56
Jam 1 dalam detik   : 44549
Jam 2 dalam detik   : 32456
Status          : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2)      : -12093detik
Jam 1 Detik Berikutnya : 12:22:30
Jam 1 Detik Sebelumnya : 12:22:28
Hasil J1 + J2          : 21:23:25

Program Selesai.
```

c) JamDriver2_NIM.java

- Menuliskan program sebagai **driver interaktif** untuk menguji kelas Jam_2511533027 dengan input dari pengguna.
- Menggunakan **kelas Scanner** untuk membaca input jam, menit, dan detik dari user.
- Program meminta pengguna untuk memasukkan **dua buah objek jam**:
  - ❖ Jam pertama (j1)
  - ❖ Jam kedua (j2)
- Input dilakukan melalui method buatJamDariInput(Scanner sc) yang:
  - ❖ Meminta input jam, menit, dan detik
  - ❖ Memvalidasi menggunakan method isValid()
  - ❖ Akan **mengulang (loop)** jika input tidak valid
- Program menampilkan data kedua jam:
  - ❖ Dalam bentuk string menggunakan toString()
  - ❖ Dalam bentuk total detik menggunakan toSeconds()
- Menggunakan method compareTo() untuk melakukan **perbandingan waktu**:
  - ❖ Jika hasil > 0 → Jam 1 lebih lambat dari Jam 2
  - ❖ Jika hasil < 0 → Jam 1 lebih awal dari Jam 2
  - ❖ Jika hasil = 0 → kedua jam sama
- Menggunakan method durasiDetik(j1, j2) untuk menghitung **selisih waktu dalam detik**
- Menggunakan method:
  - ❖ nextSecond() untuk mendapatkan waktu setelah 1 detik
  - ❖ prevSecond() untuk mendapatkan waktu sebelum 1 detik
- Menggunakan method plus() untuk melakukan **penjumlahan dua waktu**
- Scanner ditutup menggunakan input.close() setelah program selesai dijalankan.
- Program menampilkan pesan akhir:
  Program Selesai.

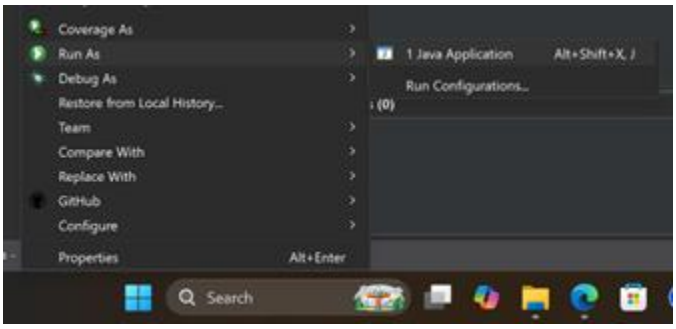
```java
package pekan1_2511533027;

import java.util.Scanner;

public class JamDriver2_2511533027 {
    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511533027 j1 = buatJamDariInput(input);

        // 2. input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511533027 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " + j1.toString());
        System.out.println("Jam 2 (String)      : " + j2.toString());
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik   : " + j2.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status           : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status           : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status           : Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)      : " + Jam_2511533027.durasiDetik(j1, j2) + "detik");

        Jam_2511533027 jNext = j1.nextSecond();
```

```java
        Jam_2511533027 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya : " + jNext);

        Jam_2511533027 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);

        // 6. Operasi Penjumlahan Jam
        Jam_2511533027 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2          : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */
    private static Jam_2511533027 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)     : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59    : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59)   : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas jam
            if (Jam_2511533027.isValid(h, m, s)) {
                return new Jam_2511533027(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
            }
        }
    }
}
```

Output:

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)     : 12
Masukkan Menit (0-59    : 56
Masukkan Detik (0-59)   : 22

--- Input Jam 2 ---
Masukkan Jam (0-23)     : 00
Masukkan Menit (0-59    : 20
Masukkan Detik (0-59)   : 09

--- Hasil Operasi ---
Jam 1 (String)      : 12:56:22
Jam 2 (String)      : 00:20:09
Jam 1 dalam detik   : 46582
Jam 2 dalam detik   : 1209
Status           : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2)      : -45373detik
Jam 1 Detik Berikutnya : 12:56:23
Jam 1 Detik Sebelumnya : 12:56:21
Hasil J1 + J2          : 13:16:31

Program Selesai.
```

6. Kompilasi dan Menjalankan Program
   a) Klik kanan pada file program, pilih **Run As**, terakhir klik **Java Application**.
   b) Lihat hasil output pada tab Console.



2.2 Analisis Hasil dan Pembahasan

1. Jam_2511533027.java
   a) Hasil: Program berhasil merepresentasikan waktu dalam bentuk objek dengan tiga komponen utama yaitu jam, menit, dan detik. Program dapat melakukan berbagai operasi seperti validasi waktu, konversi ke detik, perbandingan waktu, penjumlahan dan pengurangan waktu, serta menampilkan waktu dalam format string (HH:MM:SS).
   b) Analisis: Pada program ini sudah diterapkan konsep **OOP (Object-Oriented Programming)**, di mana data waktu disimpan dalam atribut hh, mm, dan ss yang bersifat private.

      Program juga menyediakan berbagai method untuk mengolah data tersebut, seperti toSeconds() untuk mengubah waktu menjadi detik, dan fromSeconds() untuk mengubah detik kembali menjadi jam.

      Untuk operasi seperti penjumlahan dan pengurangan waktu, program menggunakan konsep **perhitungan detik** dan **wrap 24 jam**, jadi hasilnya tetap dalam rentang waktu yang valid.

      Selain itu, method seperti compareTo() digunakan untuk membandingkan dua waktu, dan toString() membantu menampilkan waktu dengan format yang lebih mudah dibaca.
2. JamDriver_2511533027.java
   a) Hasil: Program ini menampilkan dua objek waktu yang sudah ditentukan, lalu melakukan beberapa operasi seperti penjumlahan waktu, penambahan detik, menghitung durasi, dan membandingkan kedua waktu tersebut. Semua hasilnya langsung ditampilkan di layar.

b) Analisis: Program ini berfungsi sebagai **driver** untuk mencoba semua method yang ada di kelas Jam_2511533027.

Nilai waktu dimasukkan langsung lewat konstruktor, jadi tidak perlu input dari user. Setelah itu, program memanggil beberapa method seperti plus(), nextNSeconds(), durasiDetik(), dan compareTo() untuk melihat bagaimana fungsi-fungsi tersebut bekerja.

Program ini juga menunjukkan bahwa method yang sudah dibuat bisa digunakan kembali (**reusability**), jadi kita tidak perlu menulis ulang kode yang sama.

3. JamDriver2_2511533027.java

a) Hasil: Program ini meminta pengguna untuk memasukkan dua buah waktu (jam, menit, detik). Jika input yang dimasukkan tidak sesuai, program akan meminta ulang sampai benar. Setelah itu, program menampilkan kedua waktu dalam bentuk string dan detik, lalu melakukan beberapa operasi seperti perbandingan, menghitung durasi, menampilkan waktu sebelum dan sesudah, serta hasil penjumlahan kedua waktu.

b) Analisis: Program ini merupakan pengembangan dari driver sebelumnya karena sudah menggunakan **input dari user**.

Proses input dilakukan melalui method buatJamDariInput() yang menggunakan **perulangan while** untuk memastikan data yang dimasukkan valid. Kalau user salah input, program akan terus meminta ulang sampai benar.

Selain itu, program memanfaatkan berbagai method dari kelas Jam_2511533027 seperti:

toSeconds() untuk konversi ke detik

compareTo() untuk membandingkan waktu

durasiDetik() untuk menghitung selisih waktu

nextSecond() dan prevSecond() untuk maju/mundur 1 detik

plus() untuk menjumlahkan waktu

Struktur program juga sudah rapi karena menggunakan **method tambahan** untuk memisahkan proses input, jadi lebih mudah dibaca dan dipahami.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pembuatan program **Jam** menggunakan bahasa Java sudah berhasil dengan sangat baik. Program mampu merepresentasikan waktu dalam bentuk objek yang terdiri dari jam, menit, dan detik, serta dapat melakukan berbagai operasi seperti konversi ke detik, perbandingan waktu, penjumlahan, pengurangan, dan perhitungan durasi.

Selain itu, program juga sudah menerapkan konsep **Object-Oriented Programming (OOP)** seperti enkapsulasi dan reusability, sehingga kode menjadi lebih terstruktur dan mudah dipahami. Penggunaan method dalam kelas Jam membuat setiap operasi dapat digunakan kembali tanpa harus menulis ulang kode.

Pada driver program, terutama JamDriver2, program menjadi lebih bervariasi karena sudah menerima input dari pengguna dan melakukan validasi agar data yang dimasukkan sesuai. Hal ini menunjukkan pentingnya penggunaan perulangan dan pengecekan kondisi dalam program.

Secara keseluruhan, praktikum ini sangat membantu saya memahami cara membuat program berbasis objek, mengelola data waktu, serta menerapkan konsep dasar Java dalam bentuk yang lebih nyata dan mudah dipahami.

DAFTAR PUSTAKA

[1] Tim Asisten Praktikum Struktur Data, *"Modul Praktikum Struktur Data Pekan 1: ADT,"* Universitas Andalas, 2026.

[2] Oracle, *"Java Platform, Standard Edition Documentation,"* Oracle, 2023. [Online]. Available: https://docs.oracle.com/javase/8/docs/

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“ADT”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Abstract Data Type (ADT) menggunakan studi kasus mobil. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data mobil seperti menambah, menampilkan, dan menghapus data.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR 1
DAFTAR ISI ii
BAB I 1
1.1Latar Belakang 1
BAB II 2
2.1 Kelas ADT Mobil 2
2.2 Driver 2
2.2 Kelas Driver 3
BAB III 6

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada praktikum ini dipelajari konsep Abstract Data Type (ADT) yang digunakan untuk merepresentasikan suatu objek beserta operasi yang dapat dilakukan terhadap objek tersebut. ADT membantu dalam membuat program yang lebih terstruktur dan modular.

Studi kasus yang digunakan adalah data mobil yang memiliki atribut seperti nama, tahun, cc, harga, dan merk. Program yang dibuat mampu melakukan operasi seperti menambah data, menampilkan data, serta menghapus data.

1.2 Tujuan

- Memahami konsep ADT dalam Java
- Membuat class dengan atribut dan method
- Mengimplementasikan operasi tambah dan hapus data
- Menggunakan ArrayList untuk menyimpan objek
- Membuat program terstruktur

1.3 Manfaat

- Memahami ADT dan OOP
- Membantu pengelolaan data
- Membuat program lebih rapi
- Dasar untuk program lebih kompleks

1.4 Instruksi Tugas

- Membuat ADT Mobil
- Membuat method tambah, hapus, selektor, mutator
- Membuat class ADT dan Driver

BAB II
PEMBAHASAN

2.1 Kelas ADT Mobil

Penjelasan Singkat:

Class ini digunakan sebagai ADT untuk menyimpan data mobil. Atribut dibuat untuk menyimpan informasi mobil. Konstruktor digunakan untuk menginisialisasi data. Getter digunakan sebagai selektor untuk mengambil nilai atribut.

Class ini tidak dapat dijalankan secara langsung karena tidak memiliki method main.

```java
package pekan1_2511533027;

public class ADTMobil_2511533027 {
    // atribut
    String nama;
    int tahun;
    int cc;
    long harga;
    String merk;

    // konstruktor
    public ADTMobil_2511533027(String nama, int tahun, int cc, long harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

 // selektor (getter)
    public String getNama() {
        return nama;
    }

    public int getTahun() {
        return tahun;
    }

    public int getCc() {
        return cc;
    }

    public long getHarga() {
        return harga;
    }
```

```java

    public String getMerk() {
        return merk;
    }

    // method tampil (opsional, biar gampang)
    public void tampil() {
        System.out.println("Nama Mobil: " + nama);
        System.out.println("Tahun: " + tahun);
        System.out.println("cc: " + cc);
        System.out.println("Harga: " + harga);
        System.out.println("Merk: " + merk);
    }
}

```

2.2 Kelas Driver

Penjelasan Singkat:

Class Driver digunakan untuk menjalankan program. Program menerima input dari user lalu menyimpannya ke dalam ArrayList.

Operasi yang dilakukan:

- **Tambah data** → add()
- **Tampilkan data** → loop
- **Hapus data** → remove()

```java
package pekan1_2511533027;

import java.util.ArrayList;
import java.util.Scanner;

public class Driver_2511533027 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<ADTMobil_2511533027> list = new ArrayList<>();

     // ================ TAMBAH DATA ================
        System.out.print("Masukkan jumlah mobil: ");
        int n = input.nextInt();
        input.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("\nData Mobil ke-" + (i + 1));

            System.out.print("Nama Mobil: ");
            String nama = input.nextLine();

            System.out.print("Tahun: ");
            int tahun = input.nextInt();

            System.out.print("cc: ");
            int cc = input.nextInt();

            System.out.print("Harga: ");
            long harga = input.nextLong();
            input.nextLine();

            System.out.print("Merk: ");
            String merk = input.nextLine();

            ADTMobil_2511533027 m = new ADTMobil_2511533027(nama, tahun, cc, harga, merk);
            list.add(m);
        }
```

```java
        }

        // ================= TAMPIL AWAL =================
        System.out.println("\n=== DATA MOBIL ===");
        tampil(list);

        // ================= HAPUS DATA =================
        System.out.print("\nMasukkan nomor mobil yang ingin dihapus: ");
        int hapus = input.nextInt();

        if (hapus >= 1 && hapus <= list.size()) {
            list.remove(hapus - 1);
            System.out.println("Data berhasil dihapus!");
        } else {
            System.out.println("Index tidak valid!");
        }

        // ================= TAMPIL SETELAH HAPUS =================
        System.out.println("\n=== DATA SETELAH DIHAPUS ===");
        tampil(list);
    }

    // method tampil
    public static void tampil(ArrayList<ADTMobil_2511533027> list) {
        for (int i = 0; i < list.size(); i++) {
            ADTMobil_2511533027 m = list.get(i);

            System.out.println("\nData Mobil ke-" + (i + 1));
            System.out.println("Nama Mobil: " + m.getNama());
            System.out.println("Tahun: " + m.getTahun());
            System.out.println("cc: " + m.getCc());
            System.out.println("Harga: " + m.getHarga());
            System.out.println("Merk: " + m.getMerk());
        }
    }
}
```

Sehingga Outputnya:

```
Masukkan jumlah mobil: 2

Data Mobil ke-1
Nama Mobil: toyota
Tahun: 2020
cc: 560
Harga: 340000000
Merk: avanza

Data Mobil ke-2
Nama Mobil: civic
Tahun: 2022
cc: 470
Harga: 560000000
Merk: honda

=== DATA MOBIL ===

Data Mobil ke-1
Nama Mobil: toyota
Tahun: 2020
cc: 560
Harga: 340000000
Merk: avanza

Data Mobil ke-2
Nama Mobil: civic
Tahun: 2022
cc: 470
Harga: 560000000
Merk: honda

Masukkan nomor mobil yang ingin dihapus: 2
Data berhasil dihapus!
```

```
=== DATA SETELAH DIHAPUS ===

Data Mobil ke-1
Nama Mobil: toyota
Tahun: 2020
cc: 560
Harga: 340000000
Merk: avanza
```

BAB III
PENUTUP

3.1 Kesimpulan

Pada praktikum ini, telah berhasil dibuat program ADT Mobil menggunakan Java. Program dapat melakukan operasi tambah, tampil, dan hapus data menggunakan ArrayList.

Konsep OOP seperti class, konstruktor, dan method telah diterapkan dengan baik. Selain itu, pemisahan antara class ADT dan class Driver membuat program lebih terstruktur.

DAFTAR PUSTAKA

[1] W. Wahyudi, "Abstract Data Type (ADT) dan ArrayList," Materi Praktikum Struktur Data, Universitas Andalas, 2026.

[2] Oracle Corporation, "Java Platform, Standard Edition Documentation," 2024. [Online]. Available: https://docs.oracle.com/javase/

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.